

Tìm hiểu ADO



Nội dung

- ❖ Giới thiệu ADO
- ❖ Các thành phần quan trọng của ADO
 - ❖ DataTable
 - ❖ DataSet
 - ❖ DataAdapter
 - ❖ SqlCommand
 - ❖ DataReader

Giới thiệu về ADO

- ADO là viết tắt của "ActiveX Data Objects," là một thư viện trong .NET Framework và .NET Core, có thể sử dụng trên hệ điều hành Linux và macOS. ADO.NET là một phần của .NET Core và .NET 5,6,7+.
- ADO cung cấp một tập hợp các đối tượng và thư viện để truy cập, thao tác và lưu trữ dữ liệu từ các nguồn dữ liệu khác nhau như cơ sở dữ liệu SQL Server, Oracle, hoặc các tệp dữ liệu như Excel.

Các thành phần của ADO

➤ DataTable

`DataTable` là một phần của `Dataset` và được sử dụng để lưu trữ dữ liệu dưới dạng bảng có cấu trúc. Mỗi `DataTable` có các cột và hàng, tương ứng với cấu trúc dữ liệu từ nguồn dữ liệu.

➤ Ví dụ DataTable

```
// Tạo một DataTable
DataTable dataTable = new DataTable("Employee");

// Định nghĩa cấu trúc của DataTable (các cột)
DataColumn columnID = new DataColumn("ID", typeof(int));
DataColumn columnName = new DataColumn("Name", typeof(string));
DataColumn columnEmail = new DataColumn("Email", typeof(string));

// Thêm các cột vào DataTable
dataTable.Columns.Add(columnID);
dataTable.Columns.Add(columnName);
dataTable.Columns.Add(columnEmail);
```

Các thành phần của ADO

➤ Ví dụ DataTable

Thêm dữ liệu vào DataTable

```
// Thêm dữ liệu vào DataTable
DataRow row1 = dataTable.NewRow();
row1["ID"] = 1;
row1["Name"] = "John";
row1["Email"] = "john@example.com";
dataTable.Rows.Add(row1);

DataRow row2 = dataTable.NewRow();
row2["ID"] = 2;
row2["Name"] = "Alice";
row2["Email"] = "alice@example.com";
dataTable.Rows.Add(row2);
```

```
// Hiển thị dữ liệu từ DataTable
Console.WriteLine("Danh sách nhân viên:");
foreach (DataRow row in dataTable.Rows)
{
    Console.WriteLine($"ID: {row["ID"]}, Name: {row["Name"]}, Email: {row["Email"]}");
}
```

Các thành phần của ADO

➤ DataSet

Dataset là một tập hợp các đối tượng dùng để lưu trữ dữ liệu từ nguồn dữ liệu sau khi nó đã được truy vấn và lấy về bởi DataAdapter. Nó chứa một hoặc nhiều DataTable, và cho phép bạn thao tác với dữ liệu một cách dễ dàng trong bộ nhớ.

DataSet là tập hợp của nhiều DataTable

➤ Ví dụ DataTable

```
// Tạo một DataSet
DataSet dataSet = new DataSet("MyDataSet");

// Tạo một DataTable
DataTable dataTable = new DataTable("MyDataTable");

// Định nghĩa cấu trúc của DataTable (các cột)
dataTable.Columns.Add("ID", typeof(int));
dataTable.Columns.Add("Name", typeof(string));
dataTable.Columns.Add("Age", typeof(int));

// Thêm DataTable vào DataSet
dataSet.Tables.Add(dataTable);
```

Các thành phần của ADO

➤ **DataAdapter**

`DataAdapter` là một đối tượng trong ADO.NET được sử dụng để kết nối ứng dụng với nguồn dữ liệu và thực hiện các hoạt động như truy vấn, thêm, cập nhật và xóa dữ liệu từ/nguồn dữ liệu. Nó hoạt động như một giao diện giữa `Dataset` và nguồn dữ liệu.

Trường hợp sử dụng:

SqlDataAdapter thường được sử dụng để lấy dữ liệu từ cơ sở dữ liệu và đổ vào **DataSet** hoặc **DataTable** để làm việc với dữ liệu ở dạng bảng.

Các thành phần của ADO

➤ Ví dụ sử dụng SqlDataAdapter, DataTable

```
string connectionString =  
"Server=your_server;Database=your_database;UID=your_username;Password=your_password;TrustServerCertificate=True";
```

```
// Tạo kết nối đến cơ sở dữ liệu  
using (SqlConnection connection = new SqlConnection(connectionString))  
{  
    // Mở kết nối  
    connection.Open();  
  
    // Tạo truy vấn SQL  
    string sqlQuery = "SELECT ID, Name, Email FROM Employee";  
  
    // Tạo SqlDataAdapter và DataTable  
    SqlDataAdapter dataAdapter = new SqlDataAdapter(sqlQuery, connection);  
    DataTable dataTable = new DataTable();  
  
    // Đổ dữ liệu từ cơ sở dữ liệu vào DataTable  
    dataAdapter.Fill(dataTable);  
  
    // Hiển thị dữ liệu từ DataTable  
    Console.WriteLine("Danh sách nhân viên:");  
    foreach (DataRow row in dataTable.Rows)  
    {  
        Console.WriteLine($"ID: {row["ID"]}, Name: {row["Name"]}, Email: {row["Email"]}");  
    }  
}
```


Các thành phần của ADO

➤ Ví dụ sử dụng DataAdapter, DataSet

```
string connectionString =  
"Server=your_server;Database=your_database;UID=your_username;Password=your_password;TrustServerCertificate=True";
```

```
// Tạo kết nối đến cơ sở dữ liệu  
using (SqlConnection connection = new SqlConnection(connectionString))  
{  
    // Mở kết nối  
    connection.Open();  
  
    // Tạo truy vấn SQL  
    string sqlQuery = "SELECT ID, Name, Email FROM Employee";  
  
    // Tạo SqlDataAdapter và DataSet  
    SqlDataAdapter dataAdapter = new SqlDataAdapter(sqlQuery, connection);  
    DataSet dataSet = new DataSet();  
  
    // Đổ dữ liệu từ cơ sở dữ liệu vào DataSet  
    dataAdapter.Fill(dataSet, "EmployeeData");  
  
    // Lấy DataTable từ DataSet  
    DataTable dataTable = dataSet.Tables["EmployeeData"];  
  
    // Hiển thị dữ liệu từ DataTable  
    Console.WriteLine("Danh sách nhân viên:");  
    foreach (DataRow row in dataTable.Rows)  
    {  
        Console.WriteLine($"ID: {row["ID"]}, Name: {row["Name"]}, Email: {row["Email"]}");  
    }  
}
```

Các thành phần của ADO

➤ **SqlCommand**

`SqlCommand` là một đối tượng trong ADO.NET được sử dụng để tạo và thực thi các câu lệnh SQL đến nguồn dữ liệu. Đối tượng này cho phép bạn thực hiện các thao tác như truy vấn, cập nhật, thêm và xóa dữ liệu trong cơ sở dữ liệu.

➤ **DataReader**

`DataReader` là một đối tượng trong ADO.NET được sử dụng để đọc dữ liệu từ nguồn dữ liệu một cách tuần tự. Nó cho phép bạn đọc dữ liệu một dòng tại một thay vì lấy toàn bộ dữ liệu vào bộ nhớ như `Dataset`. Điều này thường hữu ích khi bạn cần xử lý lượng dữ liệu lớn mà không muốn tốn nhiều bộ nhớ.

Trường hợp sử dụng:

DataReader hường được sử dụng để đọc dữ liệu một cách tuần tự từ cơ sở dữ liệu. Tuy nhiên, bạn có thể sử dụng SqlCommand cùng với DataReader để thực hiện truy vấn và đọc dữ liệu.

Các thành phần của ADO

➤ Ví dụ sử dụng DataReader

```
string connectionString =  
"Server=your_server;Database=your_database;UID=your_username;Password=your_password;TrustServerCertificate=True";
```

```
// Tạo kết nối đến cơ sở dữ liệu  
using (SqlConnection connection = new SqlConnection(connectionString))  
{  
    // Mở kết nối  
    connection.Open();  
  
    // Tạo truy vấn SQL  
    string sqlQuery = "SELECT ID, Name, Email FROM Employee";  
  
    // Tạo SqlCommand và SqlDataReader  
    SqlCommand command = new SqlCommand(sqlQuery, connection);  
    SqlDataReader reader = command.ExecuteReader();  
  
    // Đọc dữ liệu từ SqlDataReader  
    Console.WriteLine("Danh sách nhân viên:");  
    while (reader.Read())  
    {  
        int id = (int)reader["ID"];  
        string name = (string)reader["Name"];  
        string email = (string)reader["Email"];  
  
        Console.WriteLine($"ID: {id}, Name: {name}, Email: {email}");  
    }  
  
    // Đóng SqlDataReader  
    reader.Close();  
}
```

Các thành phần của ADO

➤ Ví dụ sử dụng SqlCommand để update dữ liệu

```
string connectionString =  
"Server=your_server;Database=your_database;UID=your_username;Password=your_password;TrustServerCertificate=True";
```

```
// Tạo kết nối đến cơ sở dữ liệu  
using (SqlConnection connection = new SqlConnection(connectionString))  
{  
    // Mở kết nối  
    connection.Open();  
  
    // Tạo truy vấn SQL cập nhật dữ liệu  
    string updateQuery = "UPDATE Employee SET Name = @Name, Email = @Email WHERE ID = @ID";  
  
    // Tạo SqlCommand  
    using (SqlCommand command = new SqlCommand(updateQuery, connection))  
    {  
        // Đặt giá trị cho các tham số  
        command.Parameters.AddWithValue("@ID", 1);  
        command.Parameters.AddWithValue("@Name", "Updated Name");  
        command.Parameters.AddWithValue("@Email", "updated.email@example.com");  
  
        // Thực thi truy vấn cập nhật  
        int rowsAffected = command.ExecuteNonQuery();  
  
        if (rowsAffected > 0)  
        {  
            Console.WriteLine("Dữ liệu đã được cập nhật thành công.");  
        }  
        else  
        {  
            Console.WriteLine("Không có bản ghi nào được cập nhật.");  
        }  
    }  
}
```

Tìm hiểu stored procedure

Một stored procedure (thường được viết tắt là "procedure") là một chương trình hoặc tập hợp các câu lệnh SQL đã được đặt tên và lưu trữ trong cơ sở dữ liệu. Mục đích chính của một stored procedure là thực hiện một loạt các tác vụ hoặc câu lệnh SQL một cách tự động và tái sử dụng được. Stored procedure thường được sử dụng trong hệ quản trị cơ sở dữ liệu (DBMS) như SQL Server, Oracle, MySQL, và PostgreSQL.

Tìm hiểu stored procedure

- **Tính năng và ứng dụng phổ biến của stored procedure**
 - **Tái sử dụng mã:** Stored procedure cho phép bạn lưu trữ các tác vụ SQL phức tạp và thường xuyên được sử dụng trong một nơi duy nhất. Điều này giúp giảm lặp lại mã và giúp bảo trì dễ dàng.
 - **Tăng hiệu suất:** Stored procedure có thể được tối ưu hóa và biên dịch sẵn trong hệ quản trị cơ sở dữ liệu, giúp tăng hiệu suất so với việc thực thi các câu lệnh SQL đơn lẻ từ ứng dụng.
 - **Bảo mật:** Bạn có thể kiểm soát quyền truy cập và thực thi stored procedure, giúp bảo mật dữ liệu và hạn chế quyền truy cập không cần thiết.

Tìm hiểu stored procedure

- **Tính năng và ứng dụng phổ biến của stored procedure**
 - **Xử lý nghiệp vụ phức tạp:** Stored procedure có thể thực hiện các tác vụ phức tạp như xử lý nghiệp vụ, tính toán, kiểm tra điều kiện, và thậm chí gửi email hoặc thông báo.
 - **Gắn kết dữ liệu:** Stored procedure thường được sử dụng để gắn kết dữ liệu từ nhiều bảng hoặc nguồn dữ liệu khác nhau thành kết quả cuối cùng để sử dụng.
 - **Tích hợp hệ thống:** Stored procedure thường được sử dụng để tích hợp dữ liệu giữa các ứng dụng khác nhau hoặc giữa hệ thống khác nhau.
 - Một số hệ quản trị cơ sở dữ liệu cho phép viết stored procedure bằng ngôn ngữ lập trình khác nhau như T-SQL (SQL Server), PL/SQL (Oracle), hoặc PL/pgSQL (PostgreSQL). Stored procedure là một phần quan trọng của phát triển ứng dụng cơ sở dữ liệu và quản lý dữ liệu trong các hệ thống thông tin doanh nghiệp.

Tìm hiểu stored procedure

➤ Ví dụ sử dụng Select, Insert, Update, Delete

```
CREATE PROCEDURE SelectEmployees
AS
BEGIN
    SELECT * FROM Employee;
END
```

```
CREATE PROCEDURE UpdateEmployee
    @ID INT,
    @Name NVARCHAR(50),
    @Email NVARCHAR(100)
AS
BEGIN
    UPDATE Employee
    SET Name = @Name, Email = @Email
    WHERE ID = @ID;
END
```

```
CREATE PROCEDURE InsertEmployee
    @Name NVARCHAR(50),
    @Email NVARCHAR(100)
AS
BEGIN
    INSERT INTO Employee (Name, Email)
    VALUES (@Name, @Email);
END
```

```
CREATE PROCEDURE DeleteEmployee
    @ID INT
AS
BEGIN
    DELETE FROM Employee
    WHERE ID = @ID;
END
```

Tìm hiểu stored procedure

➤ Ví dụ sử dụng stored procedure trong c#

```
string connectionString =
"Server=your_server;Database=your_database;UID=your_username;Password=your_password;TrustServerCertificate=True";

// Tạo kết nối đến cơ sở dữ liệu
using (SqlConnection connection = new SqlConnection(connectionString))
{
    // Mở kết nối
    connection.Open();

    // Tạo SqlCommand cho stored procedure
    using (SqlCommand command = new SqlCommand("InsertEmployee", connection))
    {
        command.CommandType = CommandType.StoredProcedure;

        // Đặt giá trị cho các tham số của stored procedure
        command.Parameters.AddWithValue("@Name", "John Smith");
        command.Parameters.AddWithValue("@Email", "john@example.com");

        // Thực thi stored procedure
        int rowsAffected = command.ExecuteNonQuery();

        if (rowsAffected > 0)
        {
            Console.WriteLine("Dữ liệu đã được thêm thành công.");
        }
        else
        {
            Console.WriteLine("Không có dữ liệu nào được thêm.");
        }
    }
}
```

Microsoft®
ASP.NET™